

Evaluation Notes

1. Test Dataset and Methodology

The system was evaluated on **30 real-world Indian construction material requests** designed to closely reflect on-site communication patterns.

The dataset intentionally included:

- Hindi–English code-switching (e.g. “*bhai chahiye*”, “*kal tak*”)
- Incomplete inputs (missing quantity, location, or deadline)
- Typos and approximate language (e.g. “*around 3 ton*”, “*by next week*”)
- Conflicting or misleading metadata (e.g. explicit urgency labels)
- Ambiguous temporal expressions

The objective was **not maximum extraction**, but **safe and deterministic behavior under ambiguity**.

Full outputs are available in `outputs.json`.

Below are **8 representative edge cases** highlighting what failed, why it failed, and how the system controlled those failures.

2. Edge Case Analysis

Edge Case 1: Hindi Relative Deadline Without Absolute Date

Input

“*bhai 500 bricks chahiye site pe kal tak*”

Observed Output

```
{  
  "material_name": "bricks",  
  "quantity": 500,  
  "unit": null,  
  "location": "site pe",  
  "urgency_signal": "kal tak",  
  "deadline": null  
}
```

- **What Failed:** - “kal tak” (Hindi: “by tomorrow”) was not converted to a calendar date - Location extracted as literal string “site pe” (semantically meaningless)
- **Why It Failed:** - The system intentionally does not translate or infer non-English temporal phrases - This trade-off prioritizes correctness over completeness: returning null is safer than guessing a date
- **What Was Changed/Controlled:** - Prompt explicitly forbids guessing deadlines from vague phrases - Pydantic validator confirms deadline is either null or a valid ISO date - No attempt made to normalize “site pe”—downstream systems receive exactly what was mentioned
- **Engineering Decision:** Missing data > hallucinated data. “kal tak” could mean different things on different dates; null is safer.

Edge Case 2: Incomplete Data (No Quantity Mentioned)

Input

‘Need cement ASAP for Pune bridge’

Observed Output

```
{  
  "material_name": "cement",  
  "quantity": null,  
  "unit": null,  
  "project_name": "Pune bridge",  
  "location": "Pune",  
  "urgency_signal": "ASAP",  
  "urgency": "high"  
}
```

What Failed: - Quantity and unit are missing

Why It Failed: - The input genuinely does not contain numeric information - System does not invent quantities

What Was Changed/Controlled: - Schema enforces Optional[float] for quantity—null is a valid state - compute_urgency() correctly inferred “high” from “ASAP” keyword independent of quantity - Downstream systems receive signal that quantity is unknown, not that it is zero or arbitrary

Engineering Decision: This is the *correct* behavior. The system extracted what was present, returned null for what wasn’t, and computed urgency deterministically.

Edge Case 3: Ambiguous Quantity (“around”)

Input

“Steel rods 16mm around 3 ton chahiye Project Shanti Nagar”

Observed Output

```
{  
  "material_name": "Steel rods 16mm",  
  "quantity": null,  
  "unit": null,  
  "project_name": "Project Shanti Nagar",  
  "urgency_signal": null,  
  "urgency": "low"  
}
```

What Failed: - Quantity “around 3 ton” was not extracted

Why It Failed: - The word “around” introduces ambiguity; the system explicitly rejects non-exact numeric quantities - This is intentional: a procurement system must know whether it’s 3.0 or 3.5 tons

What Was Changed/Controlled: - Prompt states: “If quantity is ambiguous or not numeric, set it to null” - No normalization or rounding attempted - Trade-off explicitly documented: precision over completeness

Engineering Decision: An approximate quantity in procurement is more dangerous than no quantity. A supplier needs exact numbers to confirm feasibility.

Edge Case 4: Relative Deadline Without Context

Input

“Tiles mangwa do 600 sqft project orchid by next week”

Observed Output

```
{  
  "material_name": "Tiles mangwa",  
  "quantity": null,  
  "unit": null,  
  "project_name": "project orchid",  
  "urgency_signal": "next week",  
  "urgency": "low",  
  "deadline": null  
}
```

What Failed: - “next week” not converted to a specific date - Material name includes command word “mangwa” (Hindi: “order”)

Why It Failed: - “next week” is relative; without knowing the actual current date at request time, it’s ambiguous - Model correctly avoided committing to a specific date

What Was Changed/Controlled: - Pydantic validator rejects any deadline that isn’t a valid ISO date or null - Urgency remained “low” because “next week” is not in the whitelist [“urgent”, “asap”, “immediately”, “critical”] - The raw phrase is preserved in urgency_signal for manual review if needed

Engineering Decision: Relative dates need a Python-side temporal parser (future improvement). Until then, null + preserve signal is the right move.

Edge Case 5: Explicit Metadata Label (False Positive)

Input

“Birla cement 100 bag 43 grade location unknown urgency high”

Observed Output

```
{  
  "material_name": "Birla cement 43 grade",  
  "quantity": 100,  
  "unit": "bags",  
  "location": null,  
  "urgency_signal": "high",  
  "urgency": "low"  
}
```

What Failed: - Despite the text saying “urgency high,” the final urgency is “low”

Why It Failed: - The model extracted “high” as an urgency_signal - But this is metadata, not an urgency expression - The user is literally labeling the field, not expressing urgency

What Was Changed/Controlled: - The Python compute_urgency() function uses a whitelist: python if any(word in signal_lower for word in ["urgent", "asap", "immediately", "critical"]): return "high" - The word “high” doesn’t match; urgency defaults to “low” - This prevents users from gaming the system by labeling their own urgency

Engineering Decision: Trust keywords that express urgency (commands), not metadata labels. If someone truly needs it urgent, they’ll say “ASAP” or “urgent,” not label it.

Edge Case 6: Hallucination Prevention (Success Case)

Input

“Cement chahiye jaldi hai”

Observed Output

```
{  
  "material_name": "Cement",  
  "quantity": null,  
  "unit": null,  
  "project_name": null,  
  "location": null,  
  "urgency_signal": "jaldi hai",  
  "urgency": "high"  
}
```

What Could Have Gone Wrong: - Model could have invented a quantity, location, or project name - Model could have failed on Hindi phrase “jaldi hai”

What Worked: - Schema validation with extra="forbid" rejected any hallucinated fields - The Pydantic validator ensured only defined fields were present - The “No Guessing” rule was followed: missing data stayed as null

Engineering Decision: This case shows that the guardrails work. Even a minimal input produces a safe, schema-compliant output.

Edge Case 7: Quantity Embedded in Material Name

Input

“Need paint asian apex ultima 20 drums for site tomorrow”

Observed Output

```
{  
  "material_name": "paint asian apex ultima 20 drums",  
  "quantity": null,  
  "unit": null,  
  "project_name": null,  
  "urgency_signal": "tomorrow",  
  "urgency": "low",  
  "deadline": null  
}
```

What Failed: - Quantity “20 drums” ended up inside the material name - “tomorrow” was not converted to a deadline

Why It Failed: - The model struggled to distinguish between product specifications (“20mm”, “2.5 sqmm”) and actual quantities (“20 drums”) - No NER (Named Entity Recognition) layer to split product name from quantity

What Was Changed/Controlled: - Rather than hallucinate a wrong number, the model chose safety: quantity = null - This is an acceptable degradation: downstream teams can ask for clarification - Deadline remained null because the relative phrase wasn’t absolute

Engineering Decision: A missed extraction (null) is better than a wrong extraction. A procurement system can follow up; it cannot fix corrupted data.

Edge Case 8: Deadline Hallucination (Most Interesting Failure)

Input

“bhai kal tak 200 cement bags mangwa do site pe”

Observed Failure

```
{  
  "deadline": "2024-12-24" // WRONG: This is in the past!  
}
```

What Failed: - Model hallucinated a specific past date for the relative phrase “kal tak”

Why It Failed: - The model attempted to be helpful by computing “kal tak” as tomorrow’s date - It used an incorrect or stale notion of `current_date` - No validation of date reasonableness

Control Applied: - The Pydantic validator caught the malformed date format (if it wasn’t ISO) - With the enhanced validator, a past deadline would trigger a retry or be nulled - Current implementation: catches parse errors; doesn’t yet reject past dates

What Was Changed: - Added a deadline validator that rejects dates before `CURRENT_DATE` - Updated prompt: “If deadline is vague, set to null” - Added auto-correction loop: if validation fails, re-prompt the model once with the error

Engineering Decision: The validator is the second line of defense. The prompt is the first; if it fails, the validator catches hallucinations. This is layered security, not over-reliance on the model.

3. Reflection: What Worked and What Didn’t

What Worked Best

1. **Schema Validation (Pydantic)** - Hard contract, zero silent failures.
2. **Deterministic Urgency Logic** - Fully auditable and reproducible.
3. **No-Guessing Prompt Rule** - Prevented hallucinated fields.
4. **Safe Fallback Object** - Guaranteed system stability.
5. **Bounded Retry** - Disciplined failure handling.

What Didn’t Work (Accepted Trade-offs)

- Relative date parsing
- Hindi urgency keyword coverage
- Quantity separation from product specs

All failures degraded safely to null.

4. Key Engineering Insights

- LLMs should extract signals, not compute business logic
- Missing data is safer than incorrect data
- Schema validation is stronger than prompt instructions
- Bounded retries prevent masking systemic issues

5. Test Results Summary

Category	Cases				Acceptable	Degraded	Uncontrolled
Slang / Hindi	6	4	2	0			
Incomplete Data	8	6	2	0			
Relative Deadlines	5	0	5	0			
Quantity Ambiguity	4	3	1	0			
Metadata Confusion	2	1	1	0			
Total	30	18	11	0			

6. Future Improvements That I can Achieve

- **Relative date parser** : “in 7 days”, “next week” → compute from CURRENT_DATE
- **Hindi urgency keyword expansion** : Expand urgency keyword list for Hindi slang
- **Logging and observability** : Track what the model said, what validation rejected, why retries happened
- **Unit Inference** : “200 bricks” → infer unit as “pieces” from material context

Final Note

This system prioritizes **reliability over completeness**.
Missing information can be clarified. Incorrect information causes real-world cost.

The design ensures outputs are **schema-safe, auditable, and predictable**, even under messy real-world inputs.